

Understanding Your LangGraph Reflection Agent

A plain-language, line-by-line walkthrough of chains.py and main.py

This document explains, in the simplest words possible, what every line of your two files is doing and **why** it's there.

What did you actually build this time? A self-improving writing agent. You ask it to write a tweet. Instead of just giving you one draft, it does something smarter: one AI "personality" writes a tweet, then a second AI "personality" — pretending to be a picky Instagram influencer — critiques that tweet. The critique gets fed back to the writer, who rewrites the tweet taking the feedback into account. This writer-critic-rewrite loop repeats a few times before stopping. This pattern is called **Reflection**: the AI reflects on its own work and improves it, instead of settling for its first attempt.

Unlike your last agent (which used tools like a calculator and web search), this agent uses **no tools at all**. It's just two different AI prompts talking to each other in a loop.

The two files work together like this:

- **chains.py** — defines the two AI "personalities" (the writer and the critic) and how each one is instructed to behave.
- **main.py** — wires those two personalities together into a back-and-forth loop (a flowchart), decides when to stop, and prints what the flowchart looks like.

1. chains.py — Defining the writer and the critic

This file's whole job is to set up two "chains" — a chain is just a fancy LangChain word for "a prompt template connected to an AI model, ready to be invoked." Nothing actually runs here yet; it just prepares two reusable pipelines that `main.py` will call over and over.

Line by line

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
```

- `ChatPromptTemplate` — a tool for building a reusable "template" for what gets sent to the AI, combining a fixed system instruction with dynamic parts that change each time (like the actual conversation).
- `MessagesPlaceholder` — a special slot you put inside that template which basically says "insert the real conversation history here when the template is actually used." Without it, the template would only ever send the fixed system instruction and nothing else.

```
from langchain_groq import ChatGroq
```

Same as your previous project — this is the connector that lets you talk to a model hosted on Groq's servers.

Notice there's no `load_dotenv()` in this file. That's because in this project, `load_dotenv()` is called once at the very top of **`main.py`**, before `chains.py` is even imported — so by the time `ChatGroq(...)` runs below, the API key is already loaded into the environment. You don't need to call it again here.

The critic's prompt (`reflection_prompt`)

```
reflection_prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "You are a viral instagram influencer grading a dm. Generate critique and recommendations for the user."
        "Always provide detailed recommendations, including requests for length, virality, style, etc.",
    ),
    MessagesPlaceholder(variable_name="messages"),
])
```

This builds a template for the "critic" personality.

- `ChatPromptTemplate.from_messages([...])` — you're building the template out of a list of message pieces.
- `("system", "You are a viral instagram influencer...")` — this is a tuple: the first item, "system", means "this is a system instruction, not something a user typed." The second item is the actual instruction text: play the role of a picky influencer grading a DM, and always give detailed, actionable feedback (on length, virality, style, etc.).
- Notice the two strings on separate lines with no comma between them — that's Python automatically joining adjacent string literals into one longer string. It's just a way to write one long instruction across two

lines for readability; it isn't two separate messages.

- `MessagesPlaceholder(variable_name="messages")` — this reserves a spot right after the system instruction where the actual conversation (whatever's currently in `state["messages"]`) will be inserted when this chain is invoked later. The name "messages" here has to match the key you'll pass in when calling `.invoke({"messages": ...})` in `main.py`.

The writer's prompt (generation_prompt)

```
generation_prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "You are a twitter techie influencer assistant tasked with writing excellent twitter posts."
        "Generate the best twitter post possible for the user's request."
        "If the user provides critique, respond with a revised version of your previous attempts.",
    ),
    MessagesPlaceholder(variable_name="messages"),
])
```

Same structure as above, but this template builds the "writer" personality instead: a tech-influencer-style Twitter writer whose job is to produce the best possible tweet for whatever the user asked, and — importantly — if it sees critique in the conversation, its instructions tell it to treat that as feedback and produce a revised version rather than starting from scratch or replying to the critique conversationally.

Think of it like this: both templates share the exact same `MessagesPlaceholder(variable_name="messages")` slot. That means both the writer and the critic see the *entire* conversation history so far — they're not blind to each other. The only difference between the two chains is the system instruction glued on top, which is what makes one "behave like" a writer and the other "behave like" a critic, even though nothing else about them is different.

Creating the model and the chains

```
llm = ChatGroq(model="openai/gpt-oss-120b")
```

Creates the AI model object — same model as your last project, but notice there's no `temperature=0` and no `.bind_tools(...)` this time. No tools are needed here (nothing to search or calculate), and you actually *want* some creative variation in the writing, so the default (non-zero) temperature is left as-is.

```
generate_chain = generation_prompt | llm
reflect_chain = reflection_prompt | llm
```

This is LangChain's "pipe" syntax, borrowed from the same idea as the Linux `|` pipe symbol: "take the output of the thing on the left, and feed it as input to the thing on the right." So `generation_prompt | llm` means: first build the actual prompt using the template (filling in the conversation), then immediately send that finished prompt into the `llm` to get a reply. The result, `generate_chain`, is a single object you can call `.invoke(...)` on, and it'll silently do both steps for you. Same idea for `reflect_chain`, just using the critic's template instead.

Think of it like this: generate_chain = "fill in the writer template with the conversation, then ask the AI to reply as the writer." reflect_chain = "fill in the critic template with the conversation, then ask the AI to reply as the critic." Two different personalities, same underlying model, same conversation history.

2. main.py — Wiring the writer and critic into a loop

This file builds the flowchart that makes the writer and critic talk to each other repeatedly, and decides when to stop.

Line by line

```
from typing import TypedDict, Annotated
```

- `TypedDict` — lets you define a dictionary where you specify in advance exactly which keys it will have and what type each value should be. It's used purely to describe the "shape" of your state; it doesn't add any real functionality on its own.
- `Annotated` — lets you attach extra metadata to a type hint. You'll see below it's used to tell `LangGraph` *how* to combine new messages with old ones.

```
from dotenv import load_dotenv  
load_dotenv()
```

Loads your `.env` file (API keys) — and notice this happens right at the top of the file, before `chains.py` is imported below. That ordering matters: by the time `chains.py` runs its `ChatGroq(...)` line, the environment variables it needs are already available.

```
from langchain_core.messages import BaseMessage, HumanMessage
```

- `BaseMessage` — the generic parent type that all message types (human, AI, system) are built from. Used here just to describe "a list of message-like things" in a type hint.
- `HumanMessage` — wraps a plain string as "a message that came from a human," same as in your last project.

```
from langgraph.graph import END, StateGraph
```

Same as before: `StateGraph` is the class you use to build the flowchart, and `END` is the special marker meaning "stop here."

```
from langgraph.graph.message import add_messages
```

This imports a ready-made helper function whose job is: "when new messages come in, append them onto the existing list of messages, rather than replacing the whole list." You'll see exactly where this gets used in the next line — it's the key thing that makes your growing conversation actually accumulate instead of getting overwritten each time a node runs.

```
from chains import generate_chain, reflect_chain
```

Imports the two ready-to-use chains you built in `chains.py` — the writer and the critic.

Defining the shape of the state

```
class MessageGraph(TypedDict):  
    messages: Annotated[list[BaseMessage], add_messages]
```

This defines what your "state" (the data passed between nodes) looks like — in this project, you're defining your own instead of reusing `LangGraph`'s built-in `MessagesState` like you did last time. It says:

"the state is a dictionary with one key, messages, which holds a list of BaseMessage objects."

The Annotated[list[BaseMessage], add_messages] part is the important bit: it tells LangGraph "whenever a node returns a new value for messages, don't overwrite the old list with it — instead, run it through add_messages, which appends the new message(s) onto the existing list." This is exactly why your node functions below can return just *one new message* each time, instead of having to manually re-send the entire growing conversation history themselves.

Naming the two steps

```
REFLECT = "reflect"  
GENERATE = "generate"
```

Same idea as AGENT_REASON/ACT in your previous project — plain labels stored in variables so you don't have to retype the raw strings (and risk typos) everywhere below.

The writer step

```
def generation_node(state:MessageGraph):  
    return {"messages": [generate_chain.invoke({"messages": state["messages"]})]}
```

This function is the "writer" step of the loop.

- state["messages"] — grabs the entire conversation so far (your original request, plus any past drafts/critiques).
- generate_chain.invoke({"messages": state["messages"]}) — feeds that whole history into the writer chain you built in chains.py. Remember, generate_chain = fill in the writer's template with this conversation, then send it to the AI. The AI replies with either a first draft tweet, or — if it can see critique in the history — a revised draft.
- return {"messages": [response]} — hands the new AI reply back. Because of the add_messages annotation from earlier, LangGraph appends this single new message onto the existing list rather than replacing everything.

The critic step

```
def reflect_node(state:MessageGraph):  
    res = reflect_chain.invoke({"messages": state["messages"]})  
    return {"messages": [HumanMessage(content=res.content)]}
```

This function is the "critic" step.

- reflect_chain.invoke({"messages": state["messages"]}) — same idea as above, but this time the conversation is fed into the critic's template/personality instead, so the AI replies with a critique of the most recent draft.
- res.content — pulls just the text of that critique out of the AI's response object.
- HumanMessage(content=res.content) — here's the clever (and slightly sneaky) trick: instead of adding the critique back in as an "AI message," it's wrapped as if a *human* said it. Why? Because the writer's chain is instructed to treat human-provided critique as something to revise against. By disguising the critic's feedback as a "human message," the writer chain will treat it exactly like a real user giving feedback, and will follow its instruction to produce a revised draft — without you needing any special-case

logic to tell the writer "this specific message is critique."

Think of it like this: the critic's output gets relabeled as if it came from you (the human), purely so the writer's existing instructions ("if the user provides critique, revise") kick in automatically, with zero extra code needed.

Building the flowchart

```
builder = StateGraph(state_schema=MessageGraph)
builder.add_node(GENERATE, generation_node)
builder.add_node(REFLECT, reflect_node)
builder.set_entry_point(GENERATE)
```

- `StateGraph(state_schema=MessageGraph)` — creates a new empty flowchart, this time explicitly telling it to use your custom `MessageGraph` shape (with the auto-append behavior) instead of the default one.
- `add_node(GENERATE, generation_node)` / `add_node(REFLECT, reflect_node)` — registers your two steps under the names "generate" and "reflect".
- `set_entry_point(GENERATE)` — the flowchart always starts at the writer step first (makes sense — you need a first draft before anyone can critique it).

Deciding when to stop

```
def should_continue(state:MessageGraph):
    if len(state["messages"]) > 6:
        return END
    return REFLECT
```

This is the "traffic light" for this project, called after every generate step.

- `len(state["messages"])` — simply counts how many messages are in the conversation so far (original request + every draft + every critique).
- `if len(...) > 6: return END` — once the conversation has grown past 6 messages, stop the loop and treat whatever the writer's latest draft is as the final answer. This is a simple, crude way of limiting how many writer/critic rounds happen, so the agent can't loop forever.
- `return REFLECT` — otherwise, send the draft off to the critic for feedback.

Note the difference from your previous project: last time, the stopping condition was based on *whether the AI asked for a tool*. This time there's no tool-calling at all — the stopping condition is just "has the conversation gotten long enough yet?" It's a fixed round-limit instead of the AI deciding for itself when it's done.

```
builder.add_conditional_edges(GENERATE, should_continue, {REFLECT: REFLECT, END: END}, )
builder.add_edge(REFLECT, GENERATE)
```

- `add_conditional_edges(GENERATE, should_continue, {...})` — after the generate step runs, call `should_continue` to decide where to go next: either to reflect or to END.
- `add_edge(REFLECT, GENERATE)` — a fixed, unconditional arrow: after the critic gives feedback, always go back to the writer. This is what creates the back-and-forth loop: write → critique → rewrite → critique → ... until the message count check says stop.

```
graph = builder.compile()
```

Same as before — locks in the flowchart definition and returns a runnable object.

```
print(graph.get_graph().draw_mermaid())
graph.get_graph().print_ascii()
```

Two different ways of visualizing the same flowchart, purely for your own inspection — neither of these actually runs the agent or answers any question.

- `draw_mermaid()` — prints out text in "Mermaid" diagram syntax (a popular way to describe diagrams as text) to your terminal. You could paste that text into a Mermaid viewer to see the flowchart visually, similar to the flow.png image from your last project, but here it's just printed as text rather than saved as an image.
- `print_ascii()` — prints a simple text/character-based drawing of the same flowchart directly in your terminal, so you get a quick visual without needing any external tool.

Running it

```
if __name__ == "__main__":
    print("Hello LangGraph!")
```

Think of it like this: this project's `main.py` never actually calls `graph.invoke(...)`. So even though the whole writer/critic loop is fully built and compiled, running this file right now only prints the diagram and "Hello LangGraph!" — it does not actually generate or critique any tweet. To make it do that, you'd need to add something like `graph.invoke({"messages": [HumanMessage(content="write a tweet about...")])` inside the `if __name__` block, the same way your ReAct project called `app.invoke(...)`.

3. What would happen, step by step, if you added an invoke call

Assuming you asked it to write a tweet about, say, "the benefits of learning to cook":

- 1 You'd call `graph.invoke({"messages": [HumanMessage(content="write a tweet about the benefits of learning to cook")]}). The flowchart starts at generate with just your one request in the message list.`
- 2 `generation_node` runs. The writer chain sees only your request (no critique yet), so it writes a first-draft tweet. That draft gets appended as message #2.
- 3 `should_continue` checks: only 2 messages so far, which is not more than 6, so it returns REFLECT.
- 4 `reflect_node` runs. The critic chain sees your original request AND the first draft, and writes a critique (e.g. "too generic, add a hook, keep it under 280 characters..."). That critique gets wrapped as a `HumanMessage` and appended as message #3.
- 5 Flowchart goes back to generate automatically (the fixed edge).
- 6 `generation_node` runs again. This time the writer chain sees your request, its own first draft, AND what looks like a human giving critique — so per its instructions, it produces a revised tweet. Appended as message #4.
- 7 `should_continue` checks: 4 messages, still not more than 6, returns REFLECT again.
- 8 `reflect_node` runs again, critiques the revised draft. Appended as message #5.
- 9 Back to generate. Writer revises again based on the newest critique. Appended as message #6.
- 10 `should_continue` checks: 6 messages, still not strictly greater than 6, so it returns REFLECT one more time — critique round 3 happens, appended as message #7.
- 11 Back to generate one final time, writer revises again. Now the message count is 8.
- 12 `should_continue` checks: $8 > 6$, so it returns END. The loop stops, and the very last message in the list — the writer's most recent draft — is effectively your final tweet.

Think of it like this: this whole file is a loop too, just like your last project — but instead of *think* → *use a tool* → *look at result* → *think again*, it's *write* → *get criticized* → *rewrite* → *get criticized again*, stopping once the conversation hits a fixed length instead of once the AI decides it's satisfied.

4. How this compares to your ReAct agent project

Since you just built the ReAct tool-calling agent, here's a quick side-by-side of the key differences, since both use LangGraph but solve very different problems:

	ReAct agent (previous)	Reflection agent (this one)
Purpose	Answer questions, using tools when needed	Iteratively improve one piece of writing
Number of AI personalities	One (with tools attached)	Two (writer + critic, same underlying model)
What decides to stop	Whether the AI's last reply asked for a tool	A fixed message-count limit (> 6 messages)
Tools involved	Web search, triple() calculator	None
State shape	Used LangGraph's built-in MessagesState	Custom MessageGraph TypedDict, same idea
Does it run on its own?	Yes — main.py calls app.invoke(...)	Not yet — main.py builds and prints the graph, but never calls invoke(...)